



VIT[®]
BHOPAL

NEXUS AI

AI-Powered Conversational Assistant

VITyarthi – Build Your Own Project

PROJECT REPORT

Created by

KUSHAGRA OJHA

Registration No.: **25BAI11324**

B.Tech CSE (AI & ML)

VIT Bhopal

Java • JavaFX • SQLite • Google Gemini API

Submission. College Assignment Details

Project Title	Nexus AI – AI-Powered Conversational Assistant
Assignment	VITyarthi – Build Your Own Project
Student	Kushagra Ojha
Registration No.	25BAI11324
Programme	B.Tech CSE (AI & ML)
University	VIT Bhopal
Primary Technologies	Java, JavaFX, Maven, SQLite, JUnit 5, Google Gemini REST API
GitHub	https://github.com/kushagraacodes

Project repository: github.com/kushagraacodes

. Table of Contents

1	Abstract
2	Introduction & Problem Statement
3	Objectives and Scope
4	Project Evolution: JavaGPT → Nexus AI
5	Nexus AI UI Transformation
6	Functional Requirements
7	Non-Functional Requirements
8	System Architecture
9	Workflow
10	Implementation & Module Description
11	Data Persistence & Conversation Management
12	AI Model Integration & API Key Architecture
13	Testing & Validation
14	Challenges and Solutions
15	Security Considerations
16	Project Outcome
17	Future Enhancements
18	Conclusion
19	References

1. Abstract

Nexus AI is a modular JavaFX desktop conversational assistant developed as a VITYarthi Build Your Own Project. The application provides a clean interface for sending prompts to Google Gemini, maintaining multi-turn conversation context, storing conversations locally with SQLite, selecting supported models, and handling temporary API failures through fallback logic. The project also demonstrates software-engineering practices such as layered architecture, input validation, background execution, error handling, automated testing, and Git-based project organization.

An important part of the development journey was the evolution of the project identity. The application was initially developed and tested as **JavaGPT**. After the core workflow and Gemini integration were successfully tested, the user-facing product name and visual branding were changed to **Nexus AI**. This report documents that transition so the development history and final submission identity are clear.

Keywords: Java, JavaFX, Gemini API, conversational AI, SQLite, REST API, Maven, JUnit, software architecture.

2. Introduction & Problem Statement

Conversational AI applications normally combine several software components: a user interface, an API client, conversation state, persistent storage, validation, and error handling. Building these pieces in Java provides a practical opportunity to demonstrate both programming fundamentals and software-development methodology.

Problem Statement: Develop a student-oriented desktop application that provides a simple conversational interface to a cloud AI model while demonstrating modular design, persistent data management, API integration, validation, testing, and a professional user experience.

Nexus AI addresses this problem through a JavaFX desktop interface connected to the Google Gemini REST API. SQLite is used to retain conversation history locally, while the service layer isolates AI communication from the UI.

3. Objectives and Scope

Objectives

- Build a functional conversational AI application using Java.
- Integrate Google Gemini through a REST API.
- Provide a modern JavaFX graphical user interface.
- Support multi-turn conversation context.
- Persist conversations and messages using SQLite.
- Provide model selection and retry/fallback behaviour.
- Keep the interface responsive during network operations.
- Demonstrate validation, testing, error handling, and maintainable project structure.

Scope

The submission focuses on a desktop academic prototype. It supports local conversation management and direct Gemini API communication. A secure backend-based architecture is identified as the appropriate next step for a public, multi-user deployment.

4. Project Evolution: JavaGPT → Nexus AI

The project did not begin with the final name Nexus AI. During the initial development stages, the application was called **JavaGPT**. The name was used while the main JavaFX interface, database layer, controller, AI service, Gemini integration, model handling, and testing workflow were being developed.

The development process followed a practical sequence: first build the working application, then connect and test the real AI service, validate error/fallback behaviour, and finally polish the product identity. Once the core tests were successful, the application was rebranded from **JavaGPT** to **Nexus AI**.

Stage	Name	Development Focus
1. Initial development	JavaGPT	Core JavaFX application, controller, services and storage
2. Integration	JavaGPT	Gemini API connection, model handling and conversation context
3. Validation	JavaGPT	Successful response testing, temporary-error handling and fallback
4. Final branding	Nexus AI	UI redesign, product naming and final submission identity

Important: The name change was a post-validation branding decision. The underlying tested architecture and AI workflow were retained.

5. Nexus AI UI Transformation

The final interface was deliberately changed after successful testing so that the application presents a distinct and polished identity for the college submission. The functional foundation developed under JavaGPT was retained, while the user-facing labels, title, prompts, assistant identity, and supporting controls were updated.

UI Element	Initial JavaGPT Version	Final Nexus AI Version
Application title	JavaGPT / Gemini Assistant	Nexus AI – Gemini Assistant
Assistant identity	JavaGPT	Nexus AI
Input prompt	Generic chat prompt	“Ask Nexus AI anything...”
Header branding	JavaGPT-oriented	Nexus AI + “Powered by Google Gemini”
Request status	Development-stage status	“JavaGPT is thinking...” (intentionally retained)
Controls	Core chat controls	New Chat, Delete Chat, Refresh, Model Info, Settings
Author area	Not part of initial UI	Created by Kushagra Ojha + GitHub profile link

The final UI therefore communicates two layers of the project history: **Nexus AI** is the final product identity, while “**JavaGPT is thinking...**” remains as the requested request-status text. This makes the branding change visible without hiding the original development identity.

Author identity in the UI: The sidebar now includes a dedicated author area displaying **Created by Kushagra Ojha** and a clickable GitHub profile: github.com/kushagraacodes.

6. Functional Requirements

1. Prompt entry and input validation.
2. Multi-turn conversation context.
3. Gemini response generation through REST API.
4. New, refresh/load, and delete conversation operations.
5. Model selection.
6. Automatic fallback for retryable API failures.
7. Background execution so the JavaFX interface remains responsive.
8. SQLite persistence for conversations and messages.

7. Non-Functional Requirements

Requirement	Application Approach
Usability	Clear navigation, readable chat bubbles, visible request status and dedicated controls.
Performance	Network requests run in a JavaFX background Task rather than blocking the UI thread.
Security	API key is externalized through GEMINI_API_KEY and excluded from source control.
Reliability	Retryable failures can trigger model fallback and user-facing error messages.
Maintainability	UI, controller, service, repository and model responsibilities are separated.
Resource efficiency	Recent conversation messages are used as AI context rather than unbounded history.

8. System Architecture

Nexus AI follows a layered architecture. This keeps the graphical interface independent from persistence and AI provider details, making the project easier to understand, test, and extend.

Layer	Main Component(s)	Responsibility
Presentation	ChatWindow	JavaFX layout, chat display, controls, status and dialogs
Controller	ChatController	Validation, conversation workflow and coordination
Service	AIService / GeminiService	AI abstraction, REST request, context and fallback
Data access	ChatRepository / DatabaseManager	SQLite persistence and database setup
Model	Conversation, Message, AIResult	Application data and AI response metadata

High-level flow: User → JavaFX UI → ChatController → GeminiService → Google Gemini API → AIResult → ChatController → UI, with ChatRepository/SQLite handling local conversation persistence.

9. Workflow

1. User opens Nexus AI and selects or creates a conversation.
2. The user enters a prompt and submits it.
3. ChatController validates the input and stores the user message.
4. The request is executed in a background task so the UI remains responsive.
5. GeminiService builds the request with recent conversation context and sends it to the selected Gemini model.
6. If a retryable service error occurs, the service can attempt the configured fallback model.
7. The returned AI response is wrapped in AIResult and displayed as a Nexus AI message.
8. The conversation and messages remain available through the SQLite repository.

10. Implementation & Module Description

File / Class	Role in the Project
Main.java	JavaFX application entry point and dependency wiring.
ChatWindow.java	Final Nexus AI interface, sidebar, model selector, dialogs and request status.
ChatController.java	Coordinates input validation, conversations, persistence and AI requests.
AIService.java	Abstraction that separates the UI from the AI provider implementation.
GeminiService.java	Gemini REST integration, JSON payload creation, context handling and model fallback.
AIResult.java	Carries response text and relevant AI response information.
ChatRepository.java	CRUD-style conversation/message persistence operations.
DatabaseManager.java	SQLite connection and database initialization.
Conversation.java / Message.java	Domain models representing chats and messages.
ChatControllerTest.java	JUnit 5 tests for validation and conversation workflow behaviour.

11. Data Persistence & Conversation Management

SQLite is used as the local persistence layer. Conversations and their messages are stored so that a user can continue working with previous chats rather than losing the session when the application is closed.

- **Conversation data:** identifies individual chat sessions and their titles/metadata.
- **Message data:** stores user and assistant messages associated with a conversation.
- **Repository layer:** keeps SQL/database operations outside the JavaFX UI.
- **Local-first storage:** the academic desktop prototype does not require a separate database server.

12. AI Model Integration & API Key Architecture

The application communicates with Google Gemini using its REST API. The service sends the selected model name, conversation context and the current user prompt, then parses the returned response into the application's AIResult model.

Current academic desktop setup: the application reads the API key from the **GEMINI_API_KEY** environment variable. Therefore, a separate computer running the standalone desktop build needs its own configured key.

Why the key is not embedded: hardcoding one private API key into a distributed JAR would hide the setup step, but it would not keep the key secret. A recipient could extract it and use the owner's quota. For a real multi-user deployment, the recommended architecture is **Nexus AI Desktop** → **Secure Backend** → **Gemini API**, with the private key stored only on the backend.

This separation is especially important if the project is later published for other users rather than being used only as a local academic demonstration.

13. Testing & Validation

Test Area	Expected / Observed Behaviour
Input validation	Empty/invalid prompts are handled before an API request is made.
Demo mode	Application remains usable with a clear demo response when no API key is configured.
Real Gemini response	Successful API response is displayed in the conversation UI.
Model selection	Selected supported model is used for the request.
Temporary 503 error	Temporary service failure was observed during integration testing; fallback handling was added.
Conversation management	New, refresh/load and delete operations are available.
UI responsiveness	API work runs in the background and the status indicator changes while waiting.
JUnit	Automated tests cover key controller/workflow behaviour.

Validation sequence: the project was first exercised in Demo Mode, then connected to Gemini using the configured API key. A temporary 503 response was encountered during testing, after which fallback behaviour was verified. A successful real Gemini response was then obtained, confirming the end-to-end request/response workflow.

14. Challenges and Solutions

Challenge	Solution / Learning
Maven was initially unavailable in the terminal.	Set up the local Java/Maven development environment and continued through IntelliJ.
IntelliJ did not inherit the terminal API-key environment variable.	Configured GEMINI_API_KEY in the IntelliJ Main Run Configuration.
Gemini returned a temporary 503 response.	Implemented retryable-error handling and model fallback.
Initial UI was functional but generic.	Introduced a more polished Nexus AI visual identity and additional controls.

Project name evolved during development.

Kept JavaGPT for the tested initial stage, then rebranded to Nexus AI after successful validation.

15. Security Considerations

- API credentials are treated as secrets and are not hardcoded in the submitted source.
- The .gitignore configuration excludes environment/configuration secrets and local database/build artifacts.
- For local academic use, the environment-variable approach keeps the key outside the source files.
- For public distribution, a secure backend should be introduced so end users do not receive the private Gemini API key.
- The project should never commit a real API key to GitHub.

16. Project Outcome

The final submission delivers a working JavaFX conversational assistant with a clear modular structure and a documented development journey. The project demonstrates that an AI feature can be integrated into a conventional Java desktop application while still applying software-engineering practices such as separation of concerns, persistence, testing, validation, asynchronous execution and failure handling.

The final user-facing identity is **Nexus AI**. The application also identifies its developer directly in the interface as **Kushagra Ojha**, with the GitHub profile github.com/kushagraacodes.

17. Future Enhancements

- Secure hosted backend so multiple users can use the service without configuring individual API keys.
- Streaming AI responses for a more natural conversational experience.
- Markdown and code-block rendering.
- Document and image input.
- Searchable and exportable conversation history.
- User preferences and theme customization.
- Additional AI providers through the existing AIService abstraction.
- Migration to newer Google GenAI client/API capabilities where appropriate.

18. Conclusion

Nexus AI represents the completed form of a project that began as JavaGPT. The application was first developed under the JavaGPT name so that its architecture and AI workflow could be implemented and tested. After successful functional and integration testing, the project was rebranded to Nexus AI and its interface was polished for final submission.

The final system combines JavaFX, REST-based AI integration, SQLite persistence, model fallback, asynchronous execution and JUnit-based testing into a single academic project. The documented evolution from JavaGPT to Nexus AI also demonstrates an important software-development principle: functionality can be validated first and product presentation can then be refined without discarding the tested technical foundation.

19. References

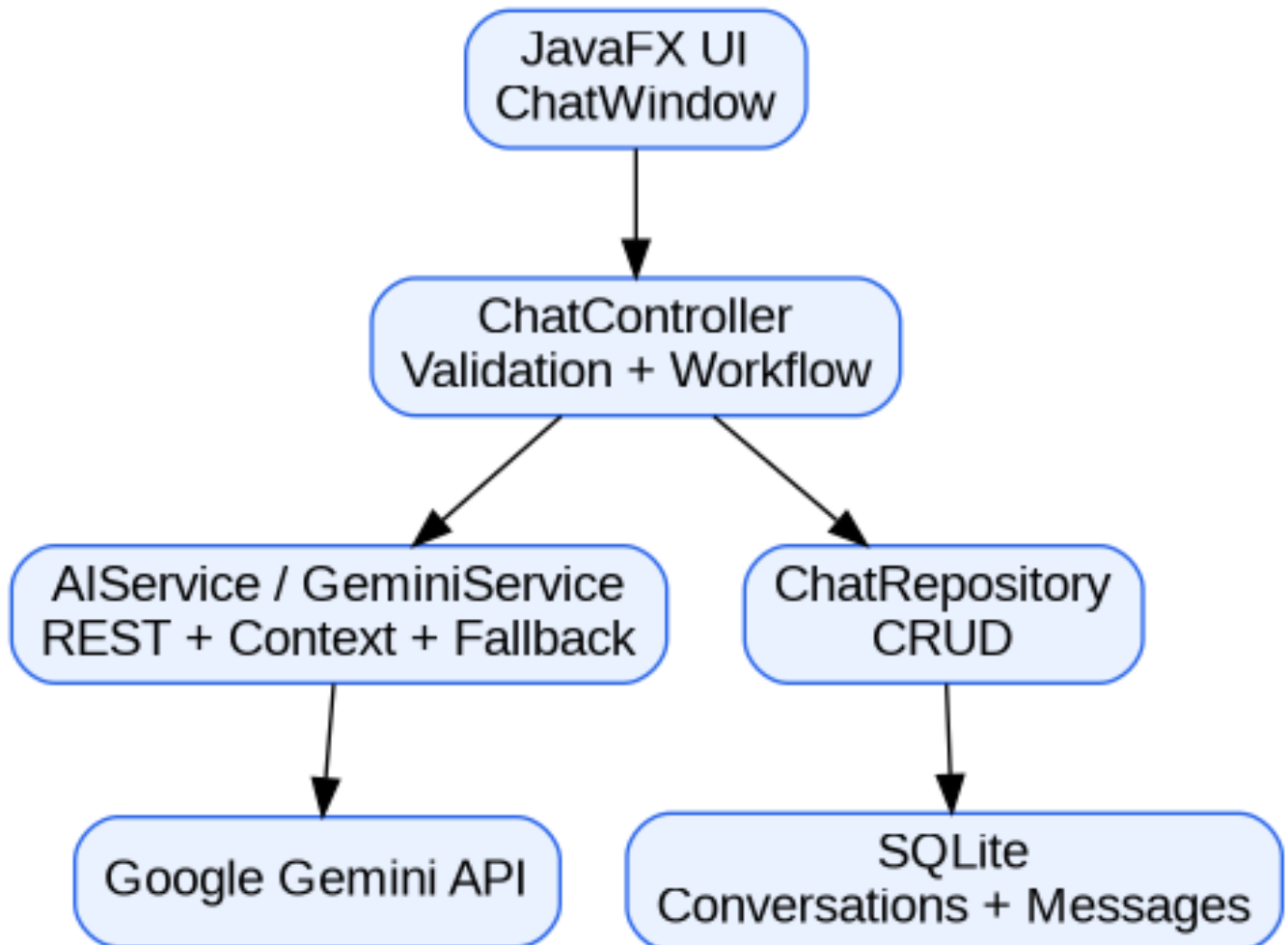
1. Google Gemini API documentation – Models and capabilities.
2. Google Gemini API documentation – Generate content / REST usage.
3. Google Gemini API documentation – API key management and security guidance.
4. JavaFX documentation – Java desktop UI framework.
5. SQLite documentation – Embedded relational database.
6. JUnit 5 documentation – Java testing framework.
7. Maven documentation – Java project build and dependency management.
8. Project GitHub profile: <https://github.com/kushagraacodes>

Declaration of project authorship: This project and the accompanying submission materials were prepared by **Kushagra Ojha (25BAI11324)** for the VITyarthi Build Your Own Project assignment.

Appendix A — Visual Design Diagrams

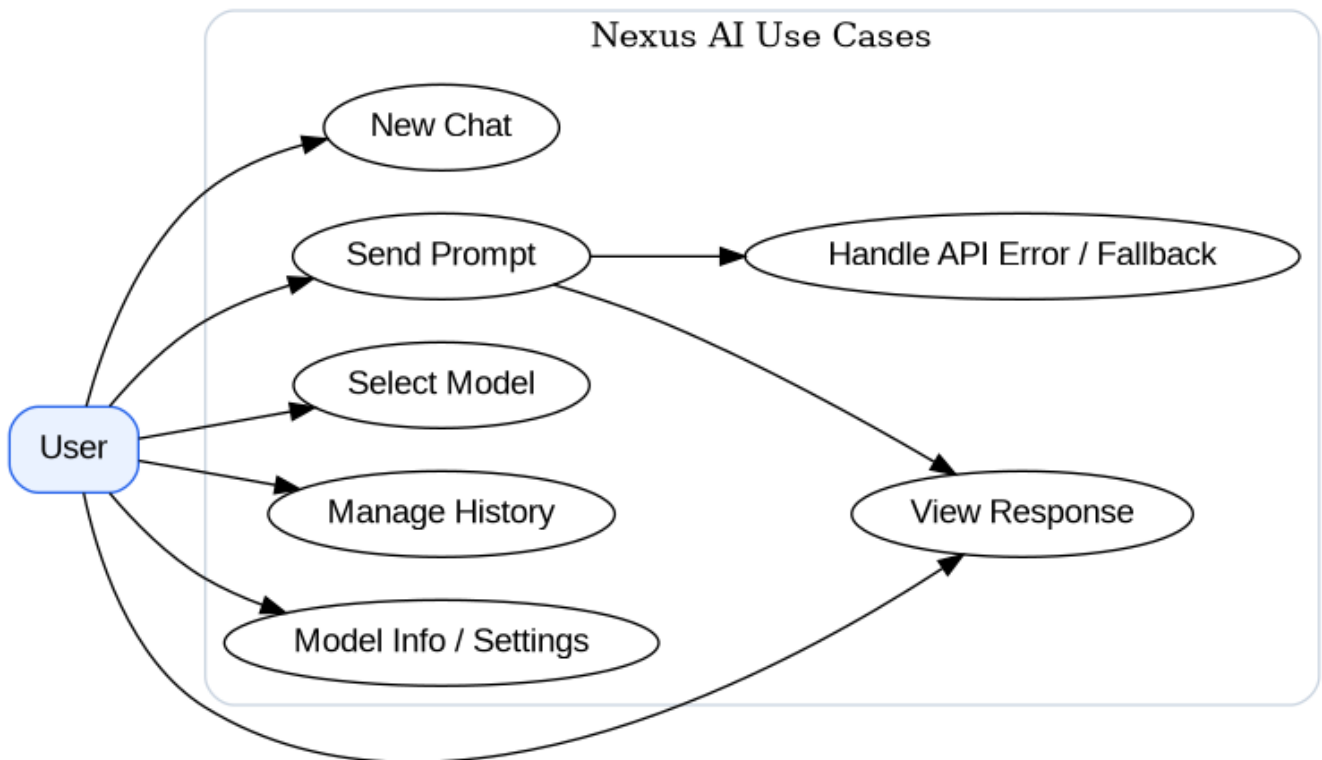
The VITyarthi project brief requires architecture, workflow and UML/design artefacts. The following visual diagrams correspond directly to the final Nexus AI source structure and are also included in the project repository under **docs/diagrams/**.

System Architecture Diagram



Shows the JavaFX presentation layer, controller, AI service, Gemini API, repository and SQLite persistence.

Use Case Diagram



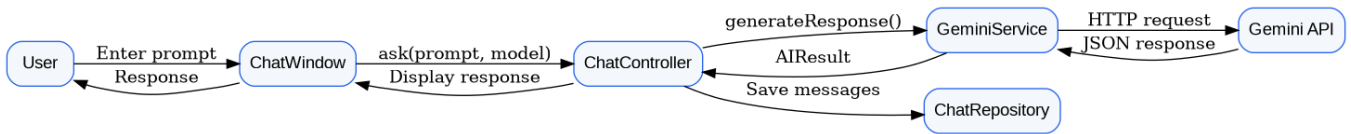
Shows the main interactions between the user and Nexus AI.

Workflow Diagram



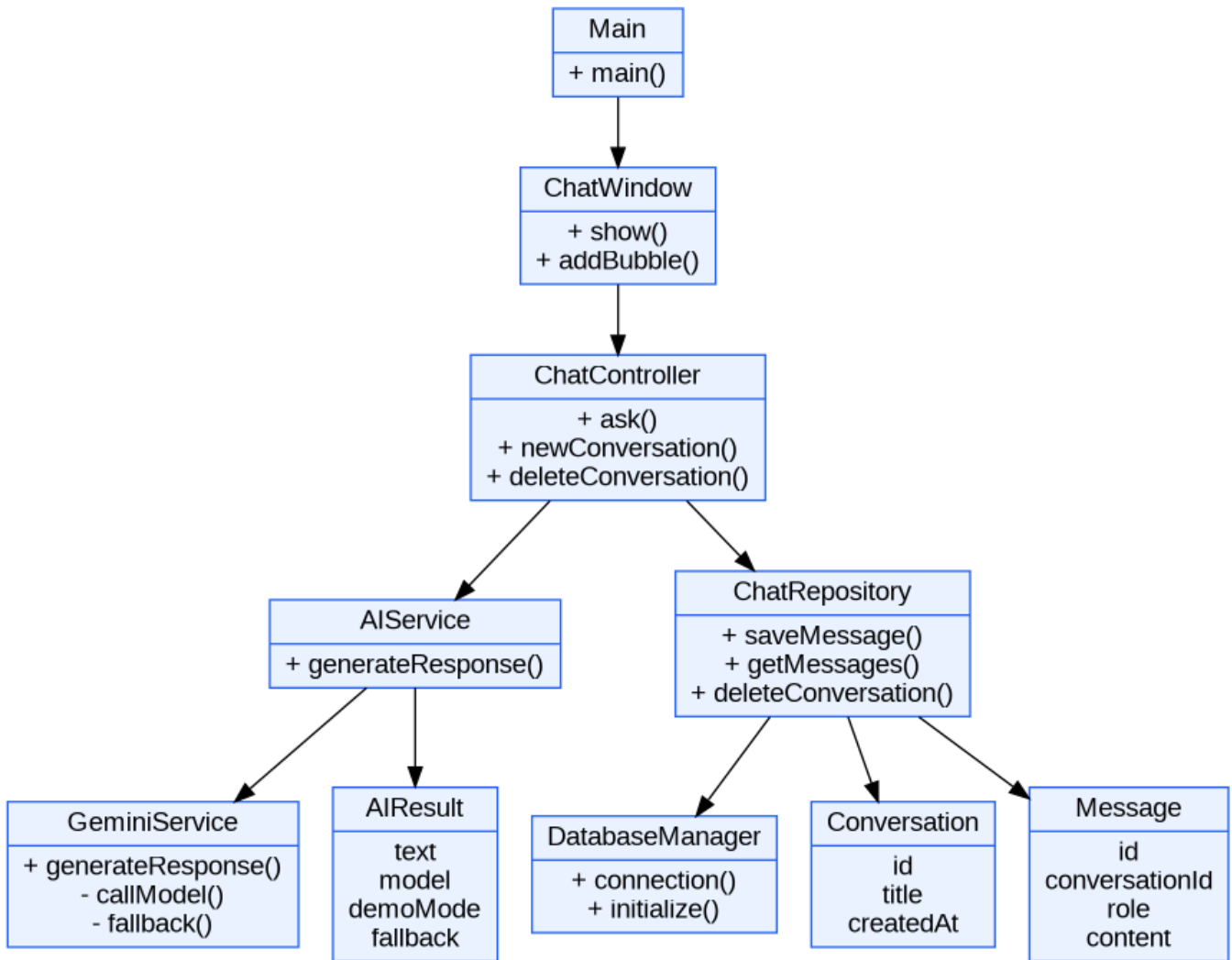
Shows the complete request workflow, including validation, API error detection and model fallback.

Sequence Diagram



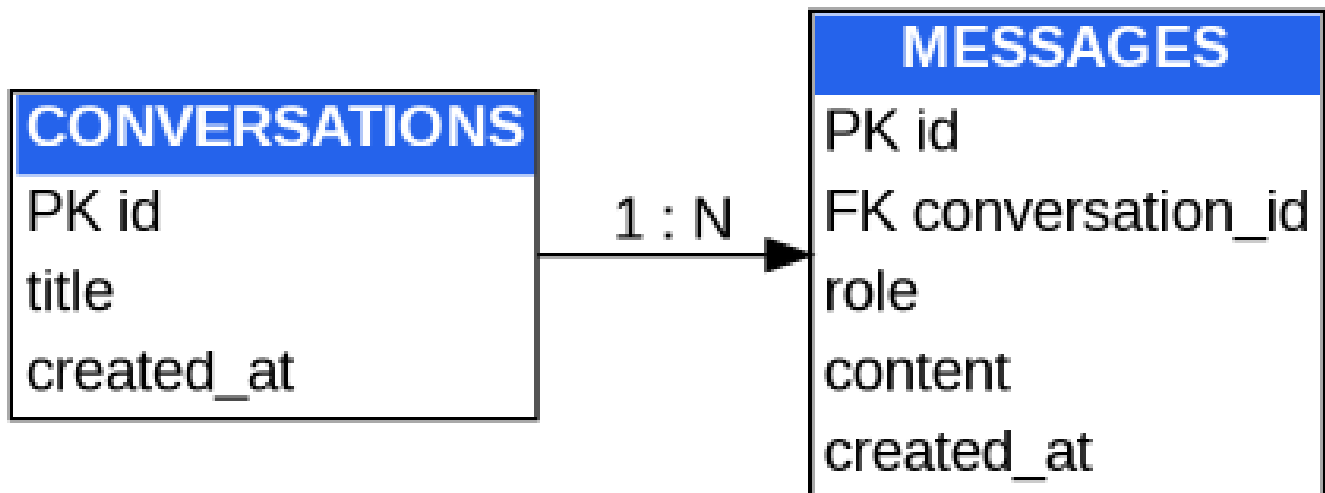
Shows the runtime interaction between the user, UI, controller, Gemini service, Gemini API and repository.

Class Diagram



Shows the final Java classes and their relationships, including GeminiService rather than the earlier OpenAI-compatible service.

ER Diagram / Storage Schema



Shows the one-to-many relationship between conversations and messages in SQLite.

Appendix B — Final Submission Checklist

Area	Final status
Functional modules	✓ 5 major modules documented and implemented
Non-functional requirements	✓ Performance, security, usability, reliability, maintainability, resource efficiency
Architecture and workflow	✓ Visual diagrams included
UML / storage diagrams	✓ Use Case, Class, Sequence and ER diagrams included
README.md	✓ Project overview, features, setup, testing, security and screenshots documented
statement.md	✓ Problem statement, scope, target users and high-level features
Source code	✓ Modular JavaFX/Maven project with 10 production classes
Testing	✓ JUnit tests + documented manual integration tests
Screenshots	✓ JavaGPT development evidence + final Nexus AI evidence
Error handling	✓ 503/429/5xx fallback strategy documented and demonstrated
Brand evolution	✓ JavaGPT → successful testing → Nexus AI documented
Author	✓ Kushagra Ojha, 25BAI11324
GitHub repository	✓ https://github.com/kushagraacodes/NexusAI

Final local verification command: `mvn clean test`. Maven was not available in the report-generation environment, so this command must be run on the student development machine before submission.

GitHub note: The repository URL was verified as an existing public repository page, but the page currently reports that the repository is empty. The final project package is prepared for upload; the source files must be pushed to that repository from the student's authenticated Git environment.

20. Development-to-Final UI Comparison

20.1 Initial Development & Testing — JavaGPT

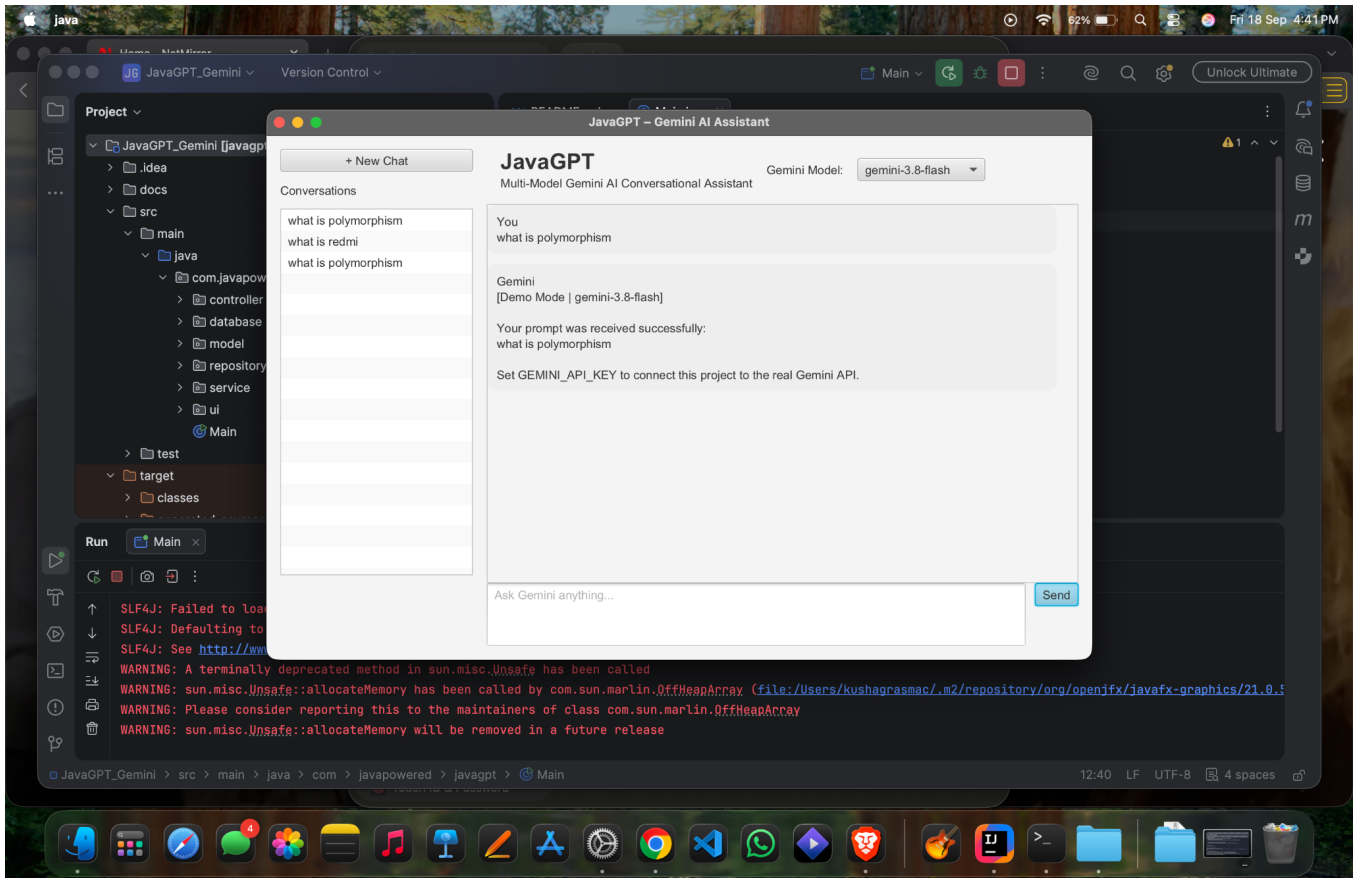
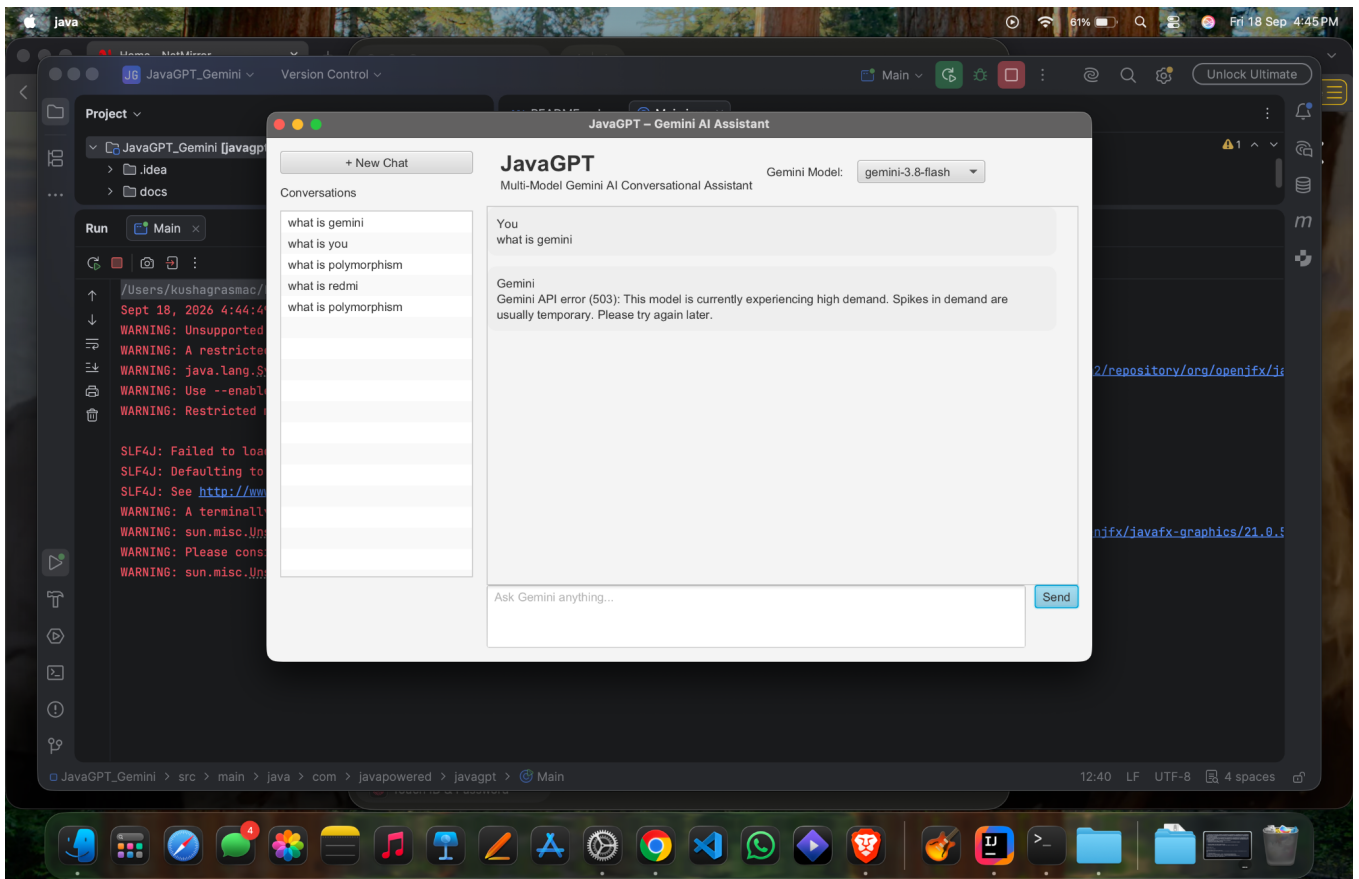


Figure 1 — Initial JavaGPT UI

This screenshot records the original JavaGPT interface used during the initial development and testing stage. It shows the JavaGPT application title, Gemini model selector, conversation history, user prompt, Demo Mode response, and the JavaFX desktop application running successfully before the final rebranding to Nexus AI.

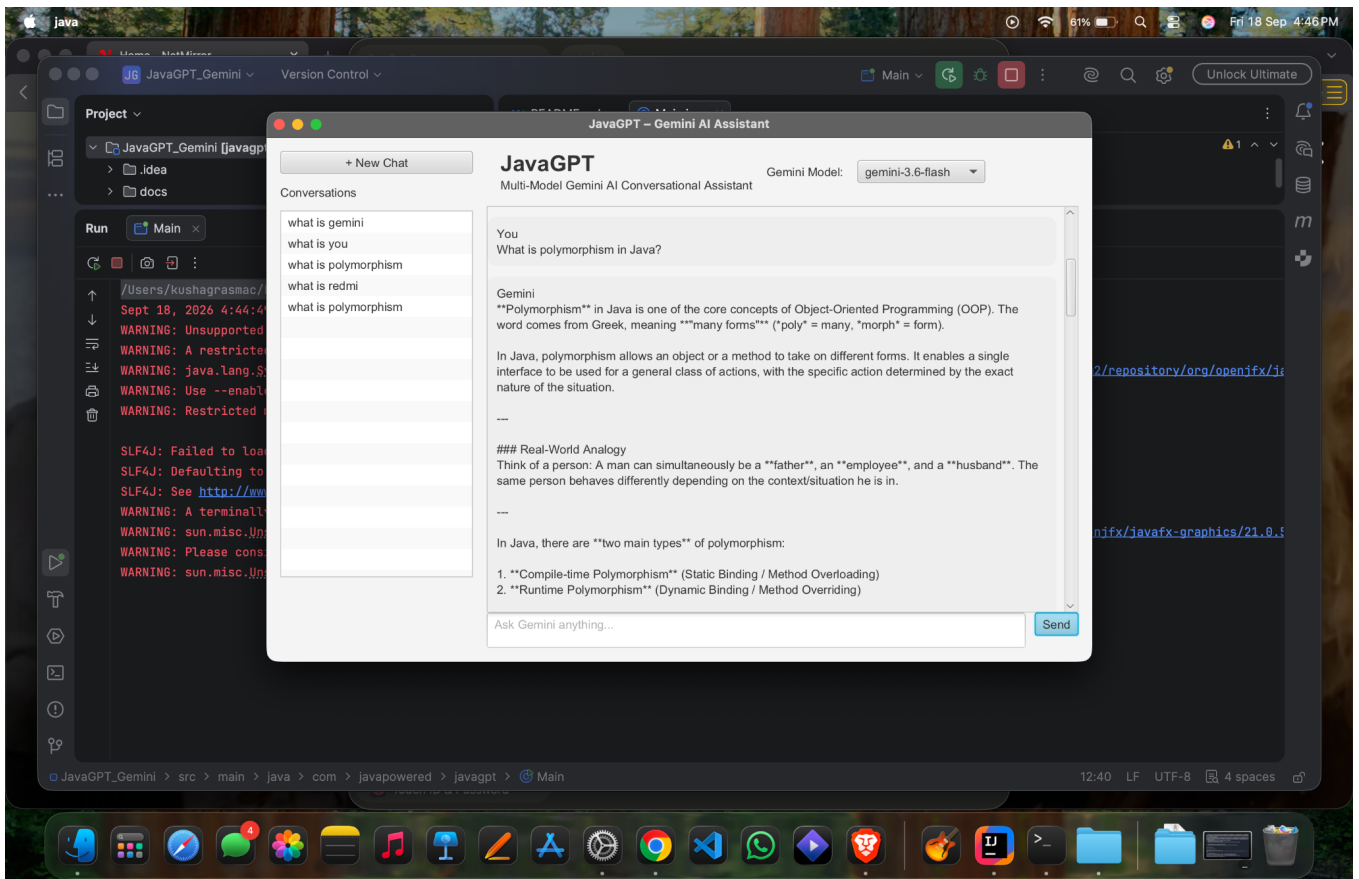
Development note: the project was initially developed and tested under the name **JavaGPT**. After the core Gemini integration and functionality were successfully tested, the project was rebranded as **Nexus AI** and the interface was refined.

Figure 2 — Initial JavaGPT: Temporary Gemini 503 Error



Development-stage JavaGPT screenshot showing a temporary Gemini service error. This evidence formed part of the integration/error-handling testing process.

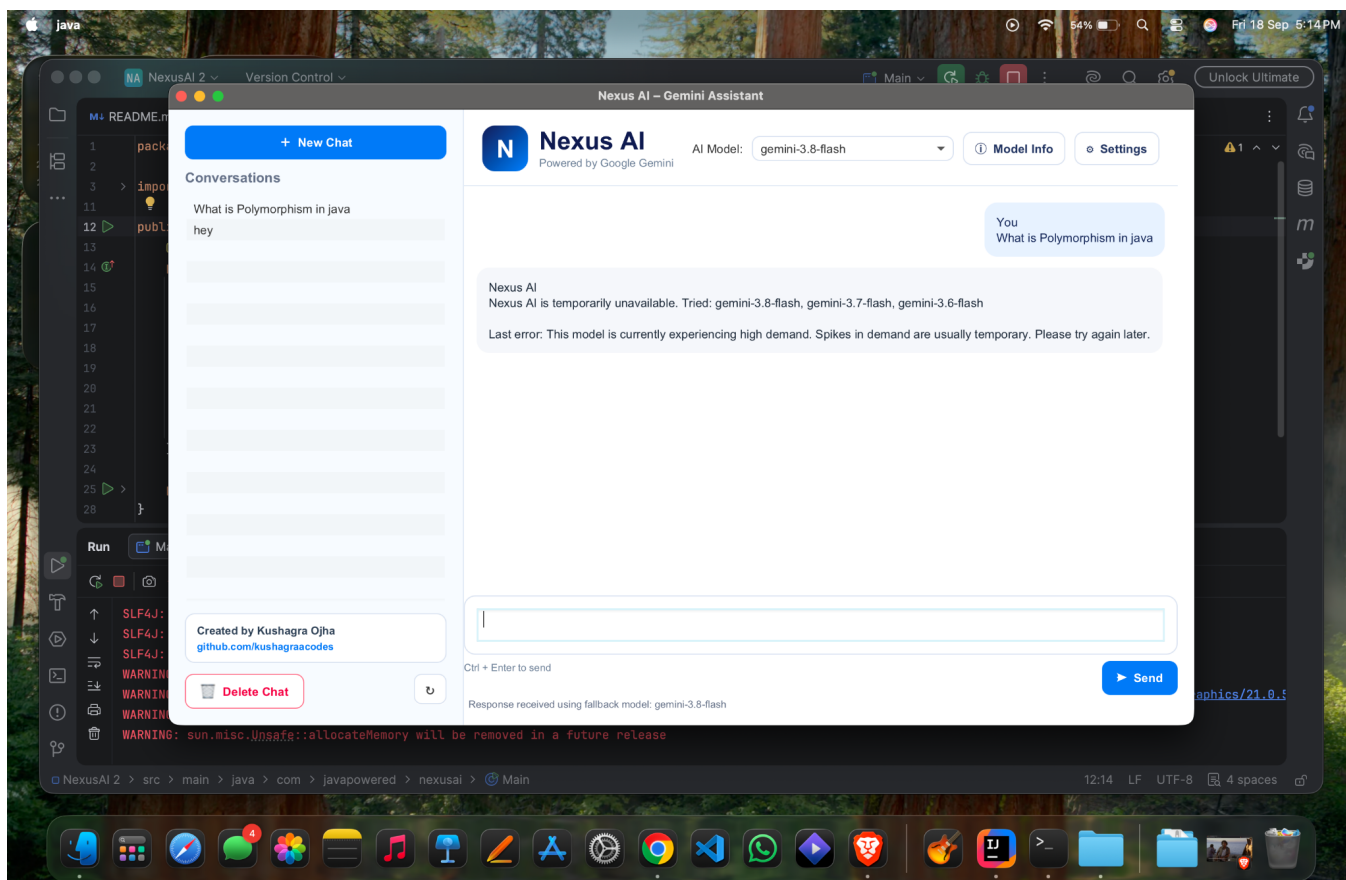
Figure 3 — Initial JavaGPT: Successful Gemini Response



Development-stage JavaGPT screenshot showing a successful real Gemini response. This demonstrates the successful end-to-end API integration before the final branding change.

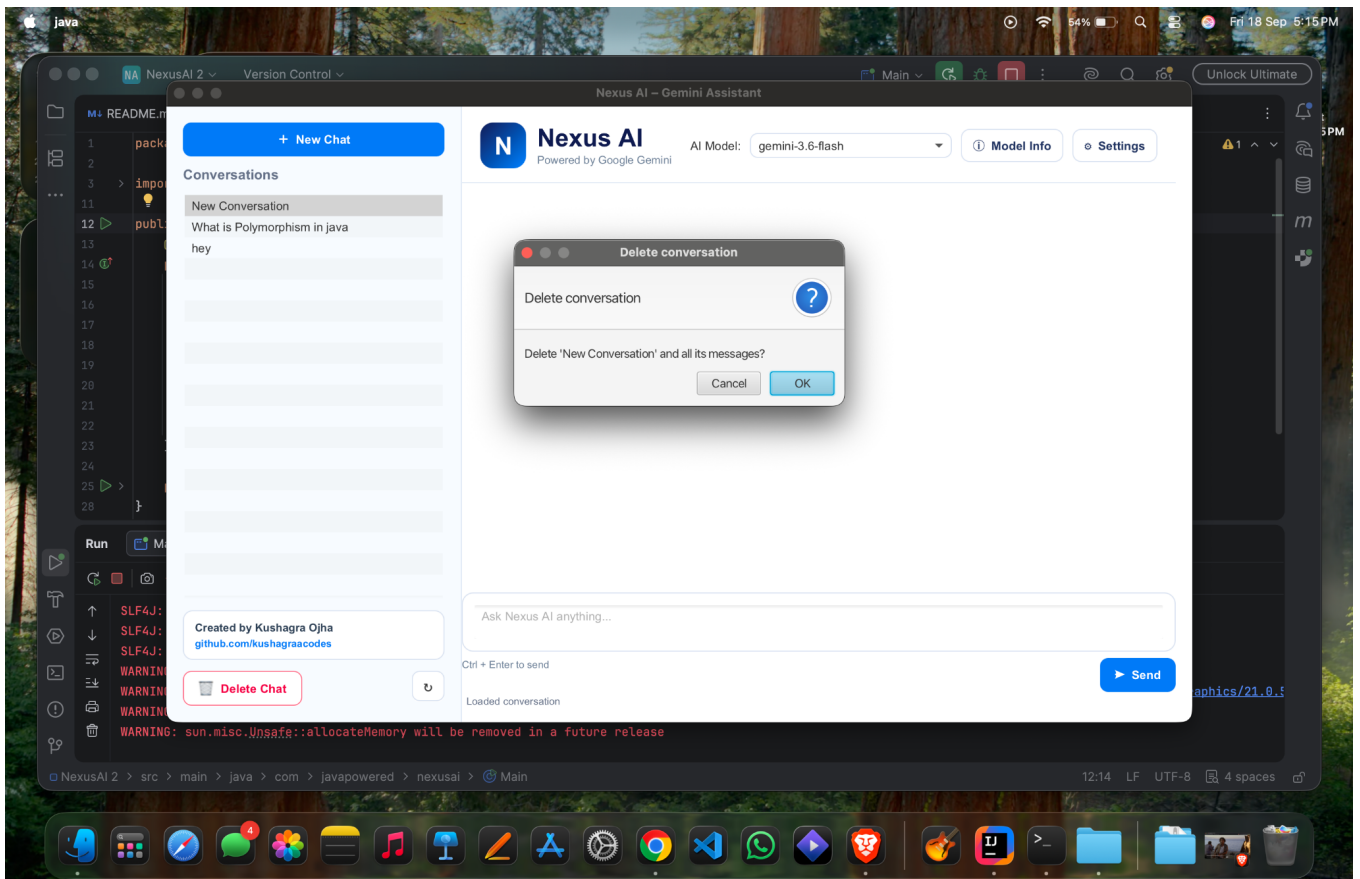
20.2 Final Interface — Nexus AI

Figure 4 — Final Nexus AI: Main Interface



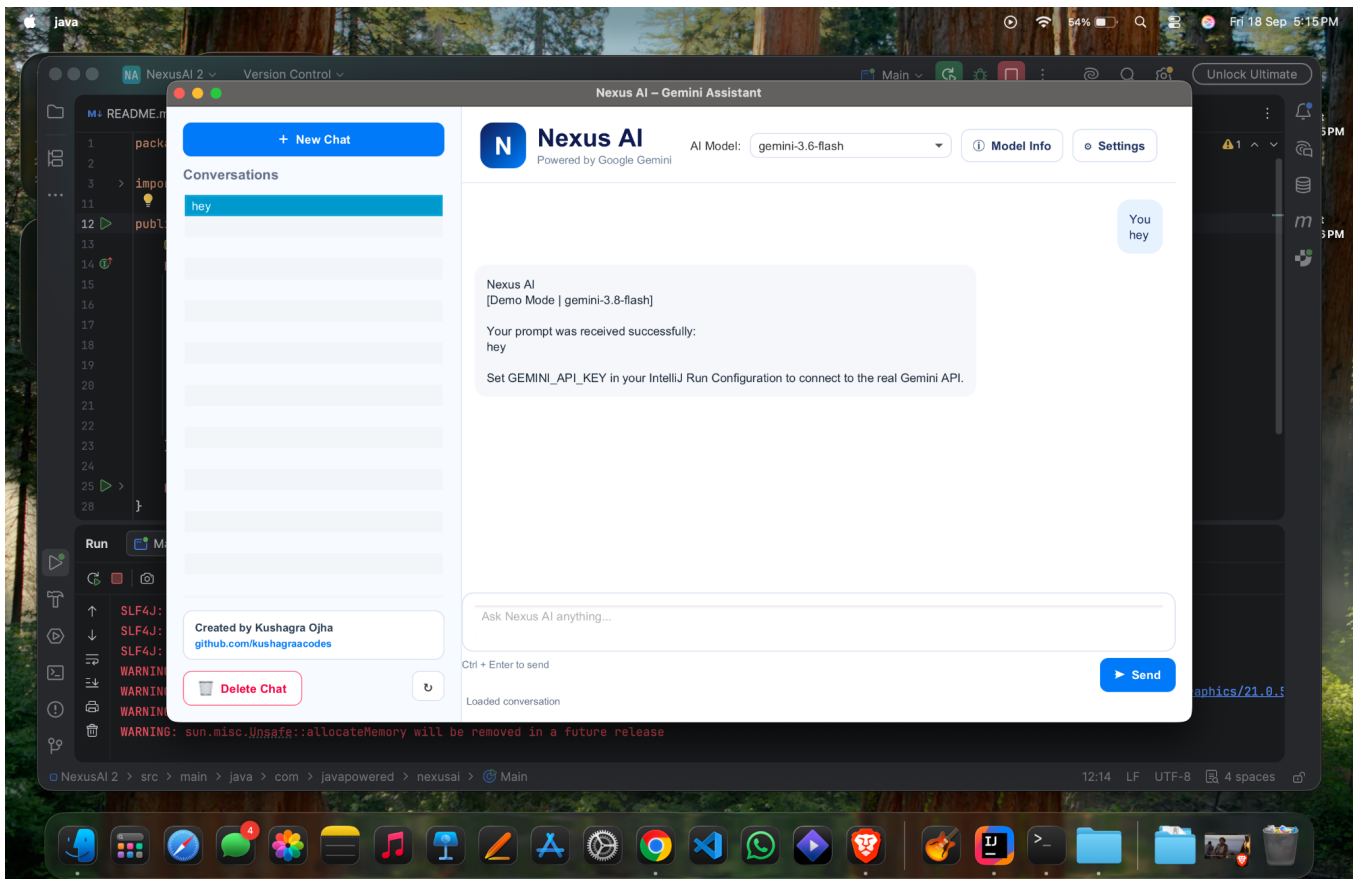
Final Nexus AI interface showing the updated branding, model selector, Model Info and Settings controls, author card, GitHub profile, conversation sidebar, and chat area.

Figure 5 — Final Nexus AI: Conversation Management



Final Nexus AI interface showing conversation management and the Delete Conversation confirmation dialog.

Figure 6 — Final Nexus AI: Demo Mode



Final Nexus AI interface operating in Demo Mode, showing the updated assistant identity, input area, response area, and project author information.

20.3 What Changed from JavaGPT to Nexus AI?

Area	Initial JavaGPT	Final Nexus AI
Project name	JavaGPT	Nexus AI
Application identity	JavaGPT-oriented interface	Nexus AI branding
Header	Earlier JavaGPT title and controls	Nexus AI + "Powered by Google Gemini"
Assistant label	JavaGPT	Nexus AI
Prompt text	Earlier generic prompt	"Ask Nexus AI anything..."
Controls	Core chat functionality	New Chat, Delete Chat, Refresh, Model Info, Settings
Author section	Not shown in the earlier screenshots	Created by Kushagra Ojha + GitHub profile
Request status	Development-stage status	"JavaGPT is thinking..." retained intentionally
Testing evidence	Demo mode, 503 error and successful Gemini response	Expanded UI and conversation-management evidence

Development history: The project was initially named JavaGPT. The earlier screenshots document its development, Demo Mode, temporary API failure, and successful Gemini integration. After these tests were completed successfully, the project was rebranded as Nexus AI and the interface was refined with the final visual identity, additional controls, author information, and GitHub profile link.

The comparison is included to make the project evolution transparent and to show that Nexus AI is the final branded version of the same tested project rather than a separate application.

21. Final Demonstration Evidence & Error Handling

This section completes the report with the latest working-screen evidence and a clear description of how the temporary Gemini API error encountered during development was handled. Together with the earlier JavaGPT and Nexus AI screenshots, it documents the complete development-to-validation journey.

21.1 Latest Successful Gemini Demonstration

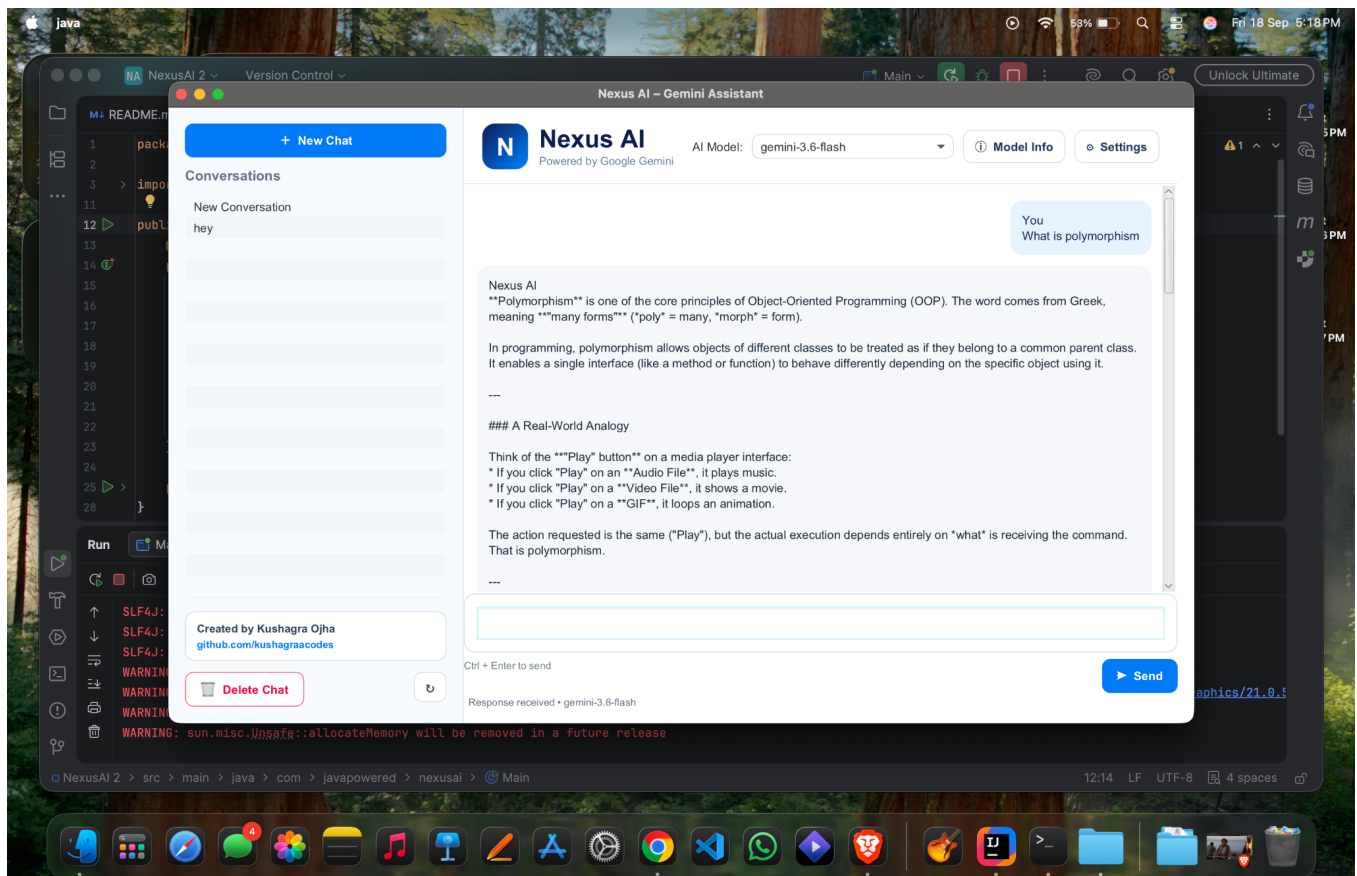


Figure 7 — Final Nexus AI: Successful Gemini response

The latest screenshot shows Nexus AI receiving the prompt “What is polymorphism” and displaying a real Gemini response using **gemini-3.6-flash**. It also shows the final Nexus AI branding, author card, GitHub link, conversation sidebar, model selector, and the response status “Response received • gemini-3.6-flash”.

This screenshot is particularly useful as final demonstration evidence because it shows the application operating end-to-end rather than only displaying a static interface.

21.2 Error Handling: Temporary Gemini API Failure

During integration testing, the application initially received a temporary **HTTP 503 Service Unavailable** response from the selected Gemini model. The error indicated temporary high demand. Instead of treating this as a permanent application failure, the project was updated to handle retryable service failures through model fallback.

Handling sequence

Step	Application Behaviour
1. Request	Nexus AI sends the user's prompt to the selected Gemini model.
2. Temporary failure	If a retryable response such as 408, 429, or selected 5xx errors occurs, the failure is recognised as potentially temporary.
3. Fallback	The Gemini service attempts the configured fallback models in sequence.
4. Successful recovery	If a fallback model responds successfully, the answer is returned to the UI and the interface records the model used.
5. Final failure	If all configured models fail, the application presents a clear Nexus AI error message rather than crashing.
6. User feedback	The request status and response area communicate the outcome to the user.

Observed during testing: A temporary 503/high-demand response was encountered while using the primary model. The application did not expose the raw failure as an unhandled crash. Fallback handling was introduced, and a later test successfully produced a real Gemini response using another configured model.

This demonstrates a practical reliability feature: the application is designed to tolerate temporary provider-side problems and attempt recovery before reporting that the service is unavailable.

21.3 Why the Error Handling Matters

AI APIs are external services, so a desktop application cannot assume that every request will succeed. Network interruptions, rate limits, temporary service unavailability, and model-specific demand can all affect a request. The fallback mechanism therefore forms part of the project's reliability design rather than being only a debugging workaround.

- **Reliability:** temporary provider failures can be recovered from without closing the application.
- **User experience:** the user receives a controlled status/error message instead of a Java exception window.
- **Maintainability:** provider-specific request logic remains inside GeminiService rather than being mixed into the JavaFX UI.
- **Testing evidence:** the earlier 503 screenshot and the later successful response screenshot together document the problem and its resolution.
- **Final result:** the latest screenshot confirms that the final Nexus AI interface can complete a real Gemini request.

21.4 Complete Development Journey

Phase	Evidence / Result
Initial JavaGPT development	Core JavaFX chat application, SQLite persistence and modular Java structure.
Gemini integration	Real API communication added and tested.
Error encountered	Temporary Gemini 503/high-demand response documented.
Error handling improvement	Retryable failures and model fallback implemented.
Successful validation	Real Gemini response successfully displayed.
Rebranding	After successful testing, JavaGPT was renamed and visually refined as Nexus AI.
Final UI	Nexus AI branding, author information, GitHub link, model controls and conversation management.

Final submission identity: Nexus AI
Developer: Kushagra Ojha
Registration No.: 25BAI11324
GitHub: github.com/kushagraacodes